Bambu Multicast Setup Guide

▼ Table of Contents

- Bambu Multicast Setup Guide
  - Supported printers
  - Requirements
    - Hardware
    - Software
    - Other
  - Let's get started
    - Phase 0 — Set up your printers
    - Phase 1 — Set up your Pi
    - Phase 2 — Install Bambu Connect
    - Phase 3 — External webcam (Optional)
    - Phase 4 — Get the printer's camera onto the local network via go2rtc
    - Phase 5 — Set up the web site
      - A. Download and install the website
      - B. Turn it into one systemd service
  - That's it!
  - Help/Contact
  - License/Copyright
  - Contributing
  - A Note on AI
  - Changelog
  - Reference links

Bambu Multicast (BM) is a companion for Bambu Lab 3D printers that lets you show multiple live streams of your printers on one web site. This project began when I got tired of needing two copies of Bambu Studio open in order to monitor jobs on both of my printers simultaneously.

Multicast is largely intended for people who want to view this on a website without having a lot of technical knowledge. The code is designed to require minimal changes -- if any -- and can be deployed and configured without any knowledge of CSS, HTML, JavaScript, or Python.

In fact, you can go through this guide and deploy without ever opening any file other than a few configs, and we will walk you through that.

> Important
>
> This document covers installing BM on a Raspberry Pi, and all commands assume the bash shell. However, any Linux-based machine will do. Windows is currently not supported, though it may be in the future.
>
> If you're of a more technical bent, I'm sure this could probably be installed on Windows using WSL, but I haven't tried it yet.

Supported printers

If you want to use your printer's built-in camera, any BambuLab printer supported by OctoEverywhere is supported by BM.

If you want to use a webcam pointed at your printer, then any webcam that can work with `go2rtc` and `ffmpeg` will do.

Requirements

> Note
>
> All links are examples. I am not affiliated or endorsed by anyone.

Hardware

- One or more BambuLab printers
- Raspberry Pi 4B or later (PiShop)
  - I use the 4gb model and my 2-printer setup works great, but more RAM allows for more printers or higher-quality streams.
- A MicroSD card that will serve as the hard drive for the Pi. (Amazon)
  - 32GB is recommended.
- (For external webcams) A webcam. (Amazon)
- The ability to write to a Micro SD card.
  - If your computer doesn't have a built-in jack, there are several adapter options out there: MicroSD to SD, SD/MicroSD to USB.

Software

- Raspberry Pi Imager so you can install an OS on the Pi, which comes without one preinstalled.

All other software is installed for free over the course of this guide.

Other

- An accocunt on OctoEverywhere.
- Admin-level access to the Pi -- we will use `sudo` a lot.

Let's get started

Phase 0 — Set up your printers

The following steps are somewhat dependent on your printer. A Google search is a better tactic here to accomplish this phase than for me to try to list the instructions for all printers and their respective menus.

1. Update your printers' firmware.
2. Enable **LAN Mode** on your printers.

> Important
>
> This will disconnect your printers from the cloud, meaning some features -- includng using the Bambu Handy app -- will no longer work. Bambu Studio will, however. For further details, see here.

3. Write down or otherwise record each printer's **Serial Number**, **Access Code**, and **IP Address**. You will need these later.
4. (Optional) Set DHCP IP reservations for your printer on your network so that the IP addresses never change.

> Tip
>
> The last step is optional but **highly recommended**. Doing this keeps the system stable in the event of a power outage, router reboot, etc. If you do not do this, then whenever your printers' IP changes, Bambu Multicast will break until you update the configuration with the new IP.

Phase 1 — Set up your Pi

> Note
>
> This document assumes you are using a Raspberry Pi 4 Model B, only because that's what I used when I created this and ran through the instructions.

1. Insert your SD card into your computer.
2. Open Raspberry Pi Imager.
3. Select **Raspberry Pi 4** and click Next.
4. Select **Raspberry Pi OS (64-bit)** and click Next.
5. Select the SD card you inserted in step 1 and click Next.
6. In the following screens, give it a hostname of your choice, enable SSH, set a username and password of your choice, and connect it to your LAN.
    - The username and password you create here will be your **Admin password** and will be used whenever logging into the Pi or running a command via `sudo`.
    - All other settings can be left as-is.
7. Complete the wizard to flash your SD card.
8. Insert the SD card into the Pi.
9. Plug the Pi in -- there is no on/off switch, so this will cause it to boot.
10. Wait 30 seconds or so, since you won't have a monitor connected and have to guess when the boot is complete.
11. Make sure `apt` is up-to-date. This is the package manager we will use.
    - Open a command-line window on your non-Pi computer.
    - Type `ssh` and press Enter.
    - Enter the username/password you created in step 6. You should now be logged into your Pi.
    - Type `sudo apt update && sudo apt full-upgrade -y` and press Enter.

- Enter your password, press Enter, and let it run. This may take a moment.

12. (Optional) Set DHCP IP reservations for your Pi on your network so that its IP address never change.

> Tip
>
> As with the printers earlier, this last step is optional but **highly recommended**. Doing this keeps the system stable in thge event of a power outage, router reboot, etc. If you do not do this, then whenever your Pi's IP changes, so will the address of the website you are setting up.

Phase 2 — Install Bambu Connect

> Important
>
> This phase will need to be repeated once for each printer you intend to use. Each time you run it, you will enter information for a different printer. You can move on to the next phases when every printer you intend to use shows up on OctoEverywhere's portal.

> Note
>
> In this step you will be installing the Bambu Connect software and telling OctoEverywhere about the printers. Don't worry if OctoEverywhere shows the printer's built-in camera when you want to use an external camera; will fix that in Phase 3, next.

1. Run the following command:

```
bash <(curl -s https://octoeverywhere.com/bambu.sh)
```

2. Follow the prompts the script walks you through.
3. Open the OctoEverywhere web portal and confirm that your printer shows up and reports its current status (e.g., temperatures, state, etc.).

Phase 3 — External webcam (Optional)

> Important
>
> This phase will need to be repeated once per external camera you use. If you are not using one, skip it. If you do this more than once, don't forget to change the port number for each one!
>
> This document assumes you are using a Logitech C920 camera only because it's the one I used when creating this. Your output may vary slightly dependinig on thge model you use.

1. Plug the webcam into the Pi.

2. Verify the camera is detected with the following commands. Your camera should show up as `/dev/video[X]` and `/dev/video[X+1]`, e.g., `/dev/video0` and `/dev/video1`.

```
sudo apt install -y v4l-utils
v4l2-ctl --list-devices
```

3. List the available stable device paths.

```
ls -l /dev/v4l/by-id
```

Your output should show something similar to the following:

```
lrwxrwxrwx 1 root root 12 Sep 20 17:19 usb-046d_HD_Pro_Webcam_C920_6C60D16F-video-index0 ->
../../video0
lrwxrwxrwx 1 root root 12 Sep 20 17:19 usb-046d_HD_Pro_Webcam_C920_6C60D16F-video-index1 ->
../../video1
```

4. You may notice that there is more than one entry for the camera in the output, as above. Some cameras have multiple streams, but only one has video (the other is metadata, which we don't need here). To determine which one you want, run this command on each of the results shown above, replacing the `-index0` with the correct entry:

```
v4l2-ctl -d /dev/v4l/by-id/usb-046d_HD_Pro_Webcam_C920_6C60D16F-video-index0 --list-formats-ext
|  grep '\[[0-9]\]'
```

Running this on my entries, I get the following output:

```
$ v4l2-ctl -d /dev/v4l/by-id/usb-046d_HD_Pro_Webcam_C920_6C60D16F-video-index0 --list-formats-
ext | grep '\[[0-9]\]'
[0]: 'YUYV' (YUYV 4:2:2)
[1]: 'H264' (H.264, compressed)
[2]: 'MJPG' (Motion-JPEG, compressed)

$ v4l2-ctl -d /dev/v4l/by-id/usb-046d_HD_Pro_Webcam_C920_6C60D16F-video-index1 --list-formats-
ext | grep '\[[0-9]\]'
$
```

The first command showed three formats available, while the second showed none. This means that the first one is the one you want. Specifically, in this example you will want `usb-046d_HD_Pro_Webcam_C920_6C60D16F-video-index0`.

Write this down. For the rest of this document, when you see `$DEVICE` use that value instead.

5. Install ustreamer

```
sudo apt update
sudo apt install -y ustreamer
```

6. Add your user to the `video` group ($USER is the username you created for the Pi)

```
sudo usermod -aG video $USER
```

7. Log out and log back into the Pi to ensure your user account now has the correct permissions.

8. (Optional, but recommended) Test everything by hand to make sure everything works, using the stable device path in step 3 for `$DEVICE`.

    1. On the Pi, run the following command:

    ```
    ustreamer --device=$DEVICE --host=0.0.0.0 --port=8081 --resolution=1280x720 --format=MJPEG
    --workers=3
    ```

    2. On a different computer, open your web browser to `http://$PI_IP:8081/stream`, where `$PI_IP` is the IP address of your PI; you should see the printer streaming.
    3. On the Pi, press CTRL+C to stop ustreamer.

> **Note**
>
> You may see a warning on the Pi: `Device doesn't support setting of HW encoding quality parameters`. You can ignore this; it's just noting that the camera's MJPEG encoder doesn't expose a software-adjustable quality control.

9. Create a file at `/etc/systemd/sudo/$NAME.service`, where `$NAME` is a unique name to identify that particular camera (e.g., `ustreamer-a1.service`, `ustreamer-p2s.service`, etc.).

```
sudo touch /etc/systemd/system/$NAME.service
```

10. Using nano (`sudo nano /etc/systemd/system/$NAME.service`), copy the text below into the file, then press CTRL+S to save and CTRL+Q to exit (don't forget to modify the text to reflect your stable device path to replace `$DEVICE`).

```
[Unit]
Description=Your description here
After=network.target

[Service]
Type=simple
ExecStart=/usr/bin/ustreamer \
    --device=/dev/v4l/by-id/$DEVICE \
    --host=0.0.0.0 \
    --port=8081 \
    --resolution=1280x720 \
    --format=MJPEG \
    --workers=3
Restart=always
RestartSec=2
```

```
User=pi
Group=video

[Install]
WantedBy=multi-user.target
```

> **Note**
>
> An explanation of a few fields:
>
> - You can change the `resolution` to taste. Higher resolution is harder on the CPU and generates more heat on the Pi; lower resolution is easier on the Pi, but lower quality. Or you can leave it as is. Consult your camera's documentation to determine what is allowed, if you want to change it at all.
> - `--port=` should have a different number for every printer you use. I suggest 8081, 8082, etc.
> - `WantedBy=multi-user.target` + `enable` (next step) is what makes it start on boot.
> - `Restart=always` restarts the service no matter *how* it exits (crash, USB hiccup, you `kill`ing it) — not just on nonzero exit codes.
> - `RestartSec=2` waits 2 seconds before each restart attempt, so a persistently failing service doesn't spin the CPU in a tight loop.

> **Important**
>
> Change `User=pi` to your actual username that you added to the `video` group, or the service will fail with a permissions error.

10. Start the service you just created

    ```
    sudo systemctl daemon-reload
    sudo systemctl enable --now ustreamer-a1.service
    ```

11. Verify the service is running correctly. The output should show `active (running)`, and the second command will show live logs; use CTRL+C to stop it and give you a command like back.

    ```
    systemctl status ustreamer-a1.service
    journalctl -u ustreamer-a1.service -f
    ```

12. Test the restart-on-failure behavior, don't just trust the config. You should see a different PID than the one you saw in step 11.

    ```
    sudo systemctl kill -s SIGKILL ustreamer-a1.service
    systemctl status ustreamer-a1.service
    ```

13. Test the start-on-boot behavior, don't just trust the config. Reboot (`sudo reboot`). When you are logged back in `systemctl is-active $NAME.service` should print `active` and the `http://$PI_IP:8081/stream` URL should

work again.

14. In the OctoEverywhere portal, open the printer's settings, enable **Alternative/External Webcam** (guide), and point it at `http://localhost:8081/stream` (Bambu Connect for the printer runs on the same Pi, so `localhost` is correct — use the Pi's LAN IP instead only if that instance runs elsewhere, and use the port number you are using for this camera).
15. Confirm the A1's camera feed in the OctoEverywhere portal is now the webcam view, at full frame rate, not the built-in camera.
16. Physically mount/aim the camera as you wish.

Phase 4 — Get the printer's camera onto the local network via go2rtc

> Important
>
> If you are not using the built-in camera for your printer(s) -- that is, you are ONLY using external cameras -- this step should be skipped. Otherwise, repeat it once per printer whose camera you are using.

> Note
>
> This section assumes a P2S for the printer, for simplicity. It's *likely* that other printers will work, but check OctoEverywhere's website for compatibility. It supports multiple printers, but if you're using one released last week it might not yet.

It turns out that Bambu printers have their own API that can be accessed to get print information, camera streams, etc. We're going to usee that.

1. Confirm "LAN Mode Liveview" is on for the P2S. This is a *separate* toggle from the general "LAN Mode" you turned on in Phase 0 — in the printer's touchscreen, find the printer's network settings and enable **LAN Mode Liveview** (sometimes just labeled "Liveview"). This is what allows a direct local video connection to the camera.

2. Sanity-check the camera URL works at all, using the P2S's LAN access code and IP address (the same ones you noted in Phase 0). The URL format Bambu printers use is:

   ```
   rtsps://bblp:<access_code>@<p2s-ip>:322/streaming/live/1
   ```

   `bblp` is always the username, for every Bambu printer.

   If you have VLC, open this URL in VLC on any computer: File → Open Network Stream → paste the URL. VLC will warn that the certificate is self-signed/untrusted — that's expected (Bambu printers use a self-signed cert here), click through it.

   Web browsers won't accept that URL, so instead you can and should test it on the Pi directly:

   ```
   sudo apt install -y ffmpeg
   ffmpeg -rtsp_transport tcp -i "rtsps://bblp:<access_code>@<p2s-ip>:322/streaming/live/1" -
   frames:v 1 -y test.jpg
   ```

You should get a jpeg of your camera view.

3. Download go2rtc onto the Pi. It's a single self-contained program -- no installer, no dependencies to install first:

> **Note**
>
> Because this isn't a package, you may need to modfy the URL below. Double check here. (Pick the first entry in the release list and use the download URL for `go2rtc_linux_arm64.`)

```
cd ~
curl -fLo go2rtc https://github.com/AlexxIT/go2rtc/releases/download/v1.9.14/go2rtc_linux_arm64
file go2rtc
```

Two safety checks are built in here. `-f` makes curl fail loudly instead of saving GitHub's "Not Found" page as if it were the program (which later shows up as `Not: command not found`). And `file go2rtc` must say `ELF 64-bit ... ARM aarch64` — if it says `x86-64`, you grabbed the PC build (`amd64`) instead of `arm64`, which fails later with `Exec format error`. Only continue once it says `aarch64`:

```
chmod +x go2rtc
sudo mv go2rtc /usr/local/bin/go2rtc
```

Confirm it's in place: `go2rtc --version` should print a version number.

4. Make sure ffmpeg is installed, if you haven't yet:

```
sudp apt install -y ffmpeg
```

5. Create its config file at `/usr/local/etc/go2rtc.yaml`:

```
sudo mkdir -p /usr/local/etc
sudo nano /usr/local/etc/go2rtc.yaml
```

Paste in (renaming `p2s_raw` and `p2s` to whatever you want, but note it for later):

```
streams:
  p2s_raw: rtspx://bblp:<access_code>@<p2s-ip>:322/streaming/live/1
  p2s: ffmpeg:p2s_raw#video=mjpeg
```

All this is doing is converting a raw video file (which the printer sends as H.264) to an MPEG through ffmpeg. Browsers can't show H.264 in an image tag, which is what we will use later, but they *can* show an MPEG. `p2s_raw` routes the data through `p2s`, which is exposed as a stream. The website will use this latter stream.

If you don't do this, go2rtc logs `codecs not matched: video:H264 => video:JPEG`.

> **Important**
>
> Replace `<access_code>` and `<p2s-ip>` with the P2S's real values. Note the scheme is **`rtspx://`**, not `rtsps://` — that `x` is go2rtc's own convention meaning "RTSP over TLS, but don't validate the certificate," which is exactly what's needed for Bambu's self-signed certificate. (Save/exit in `nano`: `Ctrl+O`, `Enter`, then `Ctrl+X`.)

6. Test it by hand first, before wiring up a system service:

```
go2rtc -config /usr/local/etc/go2rtc.yaml
```

   Leave it running, and from another machine on your LAN open `http://<pi-ip>:1984/api/stream.mjpeg?src=p2s` (modifying `p2s` to whatever you named it in step 5) — you should see the P2S's live camera feed. (`1984` is go2rtc's built-in default port; you didn't configure that, it just is one.) Once confirmed, `Ctrl+C` to stop it.

7. Create a file at `/etc/systemd/sudo/go2rtc.service`

```
sudo touch /etc/systemd/system/go2rtc.service
```

8. Using nano (`sudo nano /etc/systemd/system/go2rtc.service`), copy the text below into the file, then press CTRL+S to save and CTRL+Q to exit (don't forget to modify the text to reflect your stable device path to replace $DEVICE). See the systemd primer in Phase 3, Step 9 if any of this is unfamiliar:

```
[Unit]
Description=go2rtc camera server for P2S
After=network.target

[Service]
Type=simple
ExecStart=/usr/local/bin/go2rtc -config /usr/local/etc/go2rtc.yaml
Restart=always
RestartSec=2
User=pi

[Install]
WantedBy=multi-user.target
```

   Unlike ustreamer, no `Group=video` is needed — go2rtc isn't opening a local USB device, it's making an outbound network connection to the printer, so no special device permissions apply.

9. Load, start, and verify the new service.

```
sudo systemctl daemon-reload # Forces the system to read the .service file
sudo systemctl enable --now go2rtc.service
systemctl status go2rtc.service
```

10. Prove restart-on-failure and start-on-boot, same drill as Phase 3:

    ```
    sudo systemctl kill -s SIGKILL go2rtc.service
    systemctl status go2rtc.service
    ```

    You should get a new PID within ~2 seconds of running the kill command. To test the reboot behavior, `sudo reboot`, followed by `systemctl is-active go2rtc.service` and reloading the stream URL.

11. You should now have two independent local MJPEG streams running on the Pi:

    - The one(s) from phase 3: `http://<pi-ip>:8081/stream` (altering the port number depending on what you did), and
    - P2S (from this phase): `http://<pi-ip>:1984/api/stream.mjpeg?src=p2s`

    Worth knowing: go2rtc's connection to the P2S is completely separate from Bambu Connect's own connection to that same printer (which OctoEverywhere uses for status/AI-detection). Bambu printers handle a couple of simultaneous *local* viewers fine — this isn't the same as Bambu's *cloud* streaming, which does have a documented low concurrent-viewer limit.

Phase 5 — Set up the web site

Now we'll build the website itself. The site is mostly built for you (unless you choose to customize it), as is the small server. If you choose to go with what is provided, you will only modify config files.

> **Caution**
>
> There are a few *very* important notes to keep in mind here.
>
> - By itself, the site does not require authentication, nor do I intend to add the feature for the time being. I run this myself on a public URL, and I'm totally fine with someone seeing what I'm printing at any given moment, so I didn't worry about it.
>   - A side-effect of this is that you cannot *control* printers from this site; it is read-only. Allowing control would require authentication and user accounts, which dramatically increases complexity in this project. If there's enough demand I can add it, but until then I don't see a need.
> - HTTP/HTTPS is beyond the scope of this document. Personally, Cloudflare's tunnelling gave it to me for free, so I I didn't build it in here. If you need HTTPS and the certificates that it requires, you should consult your hosting provider.
> - If you decide to modify this, and if you want to release it to the public or fold it into this project, see the sections on copyright and contributing at the end of this document. If you decide to modify this and never make it public, enjoy!

> **Important**
>
> If you already have the site set up, back up your files first, in case you don't like the changes an update brings.

> **Note**

> This section assumes you want to host the site on the Pi itself. If you don't, the instructions *should* work on another computer if you get all the URLs right, but that has not been tested.

A. Download and install the website

1. Download the latest release from github (TODO: LINK)
2. Unzip the files to a location of your choice
3. Copy `config/config.example.json` to `config.json` in the same location

```
cp ./config/config.example.json ./config/config.json
```

4. Modify `config/config.json` using `nano ./config/config.json`. See the table below for the description fields and the changes needed for each.

Field	Description
`web_server_port`	The port the sit will sit on. Change only if desired; the default is fine.
`poll_interval_ms`	How frequently the web page will update itself from the server. This is in milliseconds, so 5000 = 5 seconds. Change to taste.
`recent_result_window_minutes`	After a print ends, how many minutes the page keeps saying "Idle (Print Succeeded)" or "Idle (Print Failed)" before going back to plain "Idle". Optional; the default is 30. Use `0` to turn the message off.
`bind_address`	Optional; the default is `0.0.0.0`, which lets anything on your network connect directly. If Cloudflare Tunnel (`cloudflared`) runs on the same machine as the site, set this to `127.0.0.1` so only the tunnel can reach it. (With `127.0.0.1`, the "reload `http://<pi-ip>:8090/`" test below only works from the Pi itself.)
`printers`	The printers that will be streamed. One entry per printer. Add or remove as necessary.
`label`	A label for the printer, which will be displayed on the page. Customize it as you want.
`camera_url`	The URL to the printer's camera that you set up earlier in this document. Must start with `http://` or `https://`.
`ip`	The printer's IP address. Note that this is **not** the address of the server streaming the cameras, it is the printer itself.
`access_code`	The access code for the printer that you wrote down at the beginning of this process.

Field	Description
`serial`	The serial number of the printer that you wrote down earlier in this process.

5. Go to the root of your site folder and make `scripts/setup.sh` executable, then run it.

```
chmod +x scripts/setup.sh
./scripts/setup.sh
```

   This will set up a virtual environment, run it, and install the bambulabs_api prerequisite. Your prompt will now show `(venv)` before it. You can type `deactivate` to exit the virtual environment, and `source venv/bin/activate` to come back. However, you must be in the virtual environment when you run this manually.

   You can now delete `scripts/setup.sh`, and for safety you probably should.

6. Test it by hand. You must be in the virtual environment when you do this (see previous step).

```
python3 server/server.py
```

   From another machine, reload `http://<pi-ip>:8090/` — the "Loading…" text should show either a live percent/layer/ETA line if a print is running or "Idle" if not. You can also check the raw data directly: `curl http://localhost:8090/status.json` should print JSON for both printers. `Ctrl+C` to stop once confirmed.

B. Turn it into one systemd service

1. Create a file at `/etc/systemd/sudo/printer-page.service`

```
sudo touch /etc/systemd/system/printer-page.service
```

2. Using nano (`sudo nano /etc/systemd/system/printer-page.service`), copy the text below into the file, then press CTRL+S to save and CTRL+Q to exit.

```
[Unit]
Description=Combined printer camera + status page
After=network.target

[Service]
Type=simple
WorkingDirectory=/home/pi/printer-page
ExecStart=/home/pi/printer-page/venv/bin/python3 /home/pi/printer-page/server/server.py
Restart=always
RestartSec=2
User=pi
```

```
[Install]
WantedBy=multi-user.target
```

Replace `/home/pi/printer-page` (it appears twice) with the real path if your username isn't `pi` and/or your folder isn't `printer-page` — run `pwd` inside that folder to confirm. Also change `printer-page` to whatever folder you unzipped the code into.

Using the venv's own `python3` binary is what makes `import bambulabs_api` work without needing that package installed system-wide.

3. Load, start, and verify it.

```
sudo systemctl daemon-reload
sudo systemctl enable --now printer-page.service
systemctl status printer-page.service
journalctl -u printer-page.service -f
```

Watch the logs for a moment for any Python errors (a typo'd IP/access code/serial would show up here as a connection error from one of the background threads).

> **Note**
>
> You probably will see some errors if everything goes well, specifically these two:
>
> - Printer Values Not Available Yet
> - Not connected to the MQTT server
>
> These are fine. They come from the printers and they're noise. You can safely ignore them.

4. Prove restart-on-failure

```
systemctl status printer-page.service # Note the PID it displays
sudo systemctl kill -s SIGKILL printer-page.service
systemctl status printer-page.service
```

You should see a different PID for the second call to `systemctl`.

5. Prove restart-on-reboot

```
sudo reboot
```

Wait a minute or two for the reboot, then reload `http://<pi-ip>:8090/`

6. Enjoy!

That's it!

You're done! You now should have a working site that streams your printers onto one page. What you do with it from here is up to you. Enjoy!

Help/Contact

If you run into any problems, please feel free to contact me on github (Merovech) or Reddit (u/pyran). If you find bugs or have suggestions for new features, please consider filing an issue on Github.

License/Copyright

This project is licensed under the MIT license. In short, you can use, change, or sell this guide and the associated software provided, as long as you keep the original author's name (i.e., me) attached to it. In other words, maintain credit where credit is due.

Contributing

I welcome contributions to the project. Go to https://github.com/Merovech/bambu-multicast to file a bug, fork the code, offer a contribution, or just say hi!

A Note on AI

This project was designed, and developed with the assistance of Claude. It was tested extensively, modified, cleaned up, and in many parts rewritten by me. I have been a software developer for over 25 years, and I heavily supervised Claude's output for safety, cleanliness, and functionality.

In other words, Claude saved me research time and coding time, but I reviewed the code and documentation before I was willing to trust it, and I did so with an experienced eye.

I've also been running it both locally and on a domain for quite a while before I even thought of packaging it up and releasing it, and it's working great so far.

Changelog

Date	Change
29 Sep 2026	Initial draft completed.
05 Oct 2026	Proofreading pass, resulting in sections being rewritten for clarity.
25 Oct 2026	Corrected the phase numbers in the reference links, removed unnecessary sections.

Reference links

- Bambu Connect Raspberry Pi setup
- External/Alternative webcam setup
- go2rtc — the local camera relay used in Phase 4 for the P2S
- bambulabs_api — the Python library used in Phase 5 for progress/layer/ETA, talking to each printer's local status feed
- OctoEverywhere API docs — no longer needed by this plan, kept only as a pointer in case you want cloud-side data later
- Cloudflare Tunnel configuration file reference